Day-71

1. **Word Ladder:-**

ask you to transform a startword into an endword, changing one letter at a time, with every intermediate word being valid (in a dictionary).

Goal is to find shortest transformation sequence.

Step 1: if endWord is not in dictionary → no solution

2: preprocess words into pattern dictionary

3: BFS initialization queue = deque([(beginword), 1)])

4: BFS traversal

a. if we reached target → return steps

b. Generate neighbours using patterns

pattern = currentword[:i] + '*' + currentword[i+1:]

c. Get all words that share this pattern

d. clear the list so we don't process it again.

pattern dict[pattern] = []

set("hit") → {'h', 'i', 't'}
set(["hit"]) → {"hit"}.

beginword = "hit"

endword = "cog"

wordList = ["hot", "dot", "dog", "lot", "log", "cog"]

Step 1:- preprocessing (pattern dictionary)

*ot → ["hot", "dot", "lot"]	l*t → [lot]
h*t → [hot, hit)	lo* → [lot, log)
ho* → [hot]	*og → [dog, log, cog]
d*t → [dot]	c*g → [cog]
do* → [dot, dog]	co* → [cog]

Step 2: BFS initialization.

queue = [(hit, 1)] visited = {"hit"}

Step 3:

iteration	Queue	currentword, level	Generate Patterns	lookups	Queue
	[(hit, 1)]	hit, 1	*it → [ ] h*t → [hot, hit] hi* → [ ]	.	[(hot, 2)]

```
def word_ladder(beginWord, endWord, wordList):
    if endWord not in wordList:
        return 0
    patterndict = defaultdict(list)
    wordList.append(beginWord)
    for word in wordList:
        for i in range(len(word)):
            pattern = word[:i] + '*' + word[i+1:]
            Patterndict[pattern].append(word)
    queue = deque([(beginWord, 1)])
    visited = set([beginWord])
    while queue:
        currentword, level = queue.popleft()
        if currentword == endword:
            return level
        for i in range(len(currentword)):
            pattern = currentword[:i] + '*' + currentword[i+1:]
            for neigh in patterndict[pattern]:
                if neigh not in visited:
                    visited.add(neighbor)
                    queue.append((neighbor, level+1))
return 0
```

2. Word Ladder II:- return all shortest paths (transformation sequences) each sequence should be returned as list of words.

```
def findLadders(beginword, endword, wordList):
    if endword not in wordlist:
        return []
    wordlist.append(beginword)
    patterndict = defaultdict(list)
    for word in wordlist:
        for i in range(len(word)):
            pattern = word[:i] + '*' + word[i+1:]
            patterndict[pattern].append(word)
    parents = defaultdict(list)  #child → list of parents.
    visited = ~~queue~~set([beginword])
    queue = deque([beginword])
    ~~found~~ = False  // to ~~stop~~ BFS at shortest level.
    while queue and notFound:
        localVisited = set()
        for _ in range(len(queue)):
            word = queue.popleft()
            for i in range(len(word)):
                pattern = word[:i] + '*' + word[i+1:]
                for neigh in patterndict[pattern]:
                    if neigh not in visited:
                        if neigh not in localvisited:
                            localVisited.add(neigh)
                            queue.append(neigh)
```

```
                parents[neighbor].append(word)
                if neighbor == endWord:
                    found = True
            visited.update(localVisited)
    if not found:
        return []
    // backtracking (dfs) to reconstruct all shortest paths from endword -> beginword
    res = []
    def dfs(word, path):
        if word == beginWord:
            res.append(path[::-1])
            return
        for p in parents[word]: // explore all parents of current word
            dfs(p, path+[p])
    dfs(endWord, [endWord])
    return res
```

Key Idea:

1. BFS → And shortest distance
2. Backtracking (DFS) → reconstruct all paths. paths
   parents[word] = all previous words that can reach it in shortest